Name: Carlos Chipantiza Neptun code: EACN90
Financial Processes (BMETE95MM14), BME, Fall 2026 course id: 54932fw

Homework 3 (Lecture 5): Digital options

Remark. The results used are stated in boxes and cited by number; results of the lectures are quoted with their cell, the few new ones are short, proved here, and have a reference (list at the end). The code shown is an exact copy of the files and every number is printed by it (files `*.out`). Files: the library of Lecture 5 (`solution` folder: `contract.py`, `market_data.py`, `model.py`, `pricer.py`, `test_pricer.py`) with the NEW CODE added at the end of `contract.py`, `pricer.py` and `test_pricer.py`; the runs `q1_digital_demo.py` (Q1) and `q2_bull_spread.py` (Q2, with `my_plotter` of Homework 1, file `plotter.py`); the notebooks `3_HW_EXE_1` (Q1) and `3_HW_EXE_2` (Q2) with the same code. After each piece of code a block diagram shows how the library grows. Running example (the market of the library): `BLUECHIP_BANK`, $S = 130.17$, $\sigma = 0.381$, $r = 0.05$; $K = 130$, $T = 1$.

1. The assignment.

Statement of the professor (`notebook_building_blocks.ipynb`, cell 19).

Create a new folder under QuantCourseBP\homeworks and copy over the content of this lecture's solution folder to start with.

1. *Get familiar with digital options and implement EuropeanDigitalContract and EuropeanDigitalAnalyticPricer (fair value and greeks) in the library. Implement unit test for fair value using pytest.* **(12 points)**
2. *Get familiar with bull spread options and by utilizing EuropeanDigitalAnalyticPricer and EuropeanAnalyticPricer, create a figure to visualize that a digital call option can be thought of as a limit of bull spread call options.* **(8 points)**

2. The problem.

To build	What it must do	Lecture 5 pattern it follows
`EuropeanDigitalContract`	store underlying, CALL/PUT, LONG/SHORT, K, T; `payoff` $= \pm\mathbf{1}\{S_T > K\}$ or $\pm\mathbf{1}\{S_T < K\}$	`EuropeanContract` (`contract.py`)
`EuropeanDigitalAnalyticPricer`	fair value and Δ, Γ, vega, Θ, ρ (closed form), times `direction`	`EuropeanAnalyticPricer` (`pricer.py`)
unit test (Q1)	fair value correct for several stocks, strikes, expiries	`TestPutCallParity` (`test_pricer.py`)
figure (Q2)	bull spreads $\to$ digital call, with both pricers	plots of Homework 1 (`plotter.py`)

The only mathematics needed: the Black–Scholes price of the digital and its five Greeks (Section 3). Everything else is code in the style of the lecture.

3. Results used.

From the lectures.

Definition 1 (Digital (cash-or-nothing) option; Hull [4], "Binary options").

With strike K and expiry T, the *digital call* pays 1 at T if $S_T > K$ and 0 otherwise; the *digital put* pays 1 if $S_T < K$:

$$V_T^{\mathrm{dc}} = \mathbf{1}\{S_T > K\}, \qquad V_T^{\mathrm{dp}} = \mathbf{1}\{S_T < K\}. \tag{1}$$

The lecture only names it ("get familiar with digital options"), so the definition is taken from [4].

Definition 2 (Bull call spread; Hull [4], "Bull spreads").

Long one call with strike K_1 and short one call with strike $K_2 > K_1$, same expiry. Payoff: $(S_T - K_1)^+ - (S_T - K_2)^+$, i.e. 0 below K_1, linear between, $K_2 - K_1$ above K_2.

Theorem 1 (Black–Scholes call and put; Lecture 3, cell 37; Lecture 5, cells 5–6).

With $\tau = T - t$ and N the standard normal cdf,

$$c = S\,N(d_1) - Ke^{-r\tau}N(d_2), \qquad p = Ke^{-r\tau}N(-d_2) - S\,N(-d_1), \tag{2}$$

$$d_1 = \frac{\ln(S/K) + (r + \sigma^2/2)\tau}{\sigma\sqrt{\tau}}, \qquad d_2 = d_1 - \sigma\sqrt{\tau}. \tag{3}$$

In the library: `EuropeanAnalyticPricer` with `calc_d1`, `calc_d2` (static).

Theorem 2 (Black–Scholes PDE; Lecture 3, cells 34–37).

The price $f(t, S)$ of every derivative on the stock satisfies

$$f'_t + \tfrac{1}{2}\sigma^2 S^2 f''_{SS} + rSf'_S - rf = 0, \tag{4}$$

with the payoff as the condition at $t = T$ ("we know the payoff at time T").

Theorem 3 (Same payoff, same price; Lecture 3, cells 28–30; Lecture 5, cell 6).

Two portfolios with the same payoff in every state have the same price today (no arbitrage); this gives put–call parity $f = c - p$. In particular 1 paid surely at T costs $e^{-r\tau}$ today (the bond in $f = S - Ke^{-r\tau}$).

New, short results (proved here; references [4], [5]).

Lemma 1 (Dominance; from Theorem 3 (no arbitrage); [4], [5]).

If two payoffs satisfy $V_T^A \geq V_T^B$ in every state, then their prices satisfy $V^A \geq V^B$.

Proof. If $V^A < V^B$: sell B, buy A, keep $V^B - V^A > 0$; at T the position pays $V_T^A - V_T^B \geq 0$. A sure gain at no risk is an arbitrage, excluded in Lecture 3 (cell 33). □

Lemma 2 (A Black–Scholes identity; algebra on (3)).

With $\varphi = N'$ the normal density,

$$S\,\varphi(d_1) = Ke^{-r\tau}\,\varphi(d_2). \tag{5}$$

Proof. By levels, using $d_2 = d_1 - \sigma\sqrt{\tau}$ and (3):

$$\begin{aligned}
\frac{\varphi(d_2)}{\varphi(d_1)} &= e^{(d_1^2 - d_2^2)/2} && \text{(definition of } \varphi\text{)} \\
&= e^{(d_1 - d_2)(d_1 + d_2)/2} && \text{(difference of squares)} \\
&= e^{\sigma\sqrt{\tau}\,(2d_1 - \sigma\sqrt{\tau})/2} && (d_1 - d_2 = \sigma\sqrt{\tau}) \\
&= e^{\ln(S/K) + r\tau} && \text{(insert } d_1 \text{ from (3))} \\
&= \frac{S}{Ke^{-r\tau}}. && \square
\end{aligned}$$

Lemma 3 (Strike derivatives; from Theorem 1 and Lemma 2; Breeden–Litzenberger [6]).

$$\frac{\partial c}{\partial K} = -e^{-r\tau} N(d_2), \qquad \frac{\partial p}{\partial K} = e^{-r\tau} N(-d_2). \tag{6}$$

Proof. Both d_1 and d_2 depend on K with $\partial d_1/\partial K = \partial d_2/\partial K$. Differentiate (2):

$$\begin{aligned}
\frac{\partial c}{\partial K} &= S\varphi(d_1)\frac{\partial d_1}{\partial K} - e^{-r\tau}N(d_2) - Ke^{-r\tau}\varphi(d_2)\frac{\partial d_2}{\partial K} && \text{(product and chain rule)} \\
&= -e^{-r\tau}N(d_2). && \text{(the two } \varphi \text{ terms cancel by (5))}
\end{aligned}$$

The put is the same computation (or parity: $c - p = S - Ke^{-r\tau}$ gives $\partial_K c - \partial_K p = -e^{-r\tau}$). $\square$

Lemma 4 (Digital prices; from Lemmas 1, 3 and Definitions 1–2; [4], [6]).

$$C_d = e^{-r\tau} N(d_2), \qquad P_d = e^{-r\tau} N(-d_2), \qquad C_d + P_d = e^{-r\tau}. \tag{7}$$

Proof. For $h > 0$ take the two bull spreads of Definition 2, scaled by $1/h$, around K: R_h long K / short $K + h$, and L_h long $K - h$ / short K. Their payoffs are piecewise linear and satisfy, for every S_T,

$$\tfrac{1}{h}\big[(S_T - K)^+ - (S_T - K - h)^+\big] \;\le\; \mathbf{1}\{S_T > K\} \;\le\; \tfrac{1}{h}\big[(S_T - K + h)^+ - (S_T - K)^+\big].$$

By Lemma 1 the prices keep the order:

$$\frac{c(K) - c(K + h)}{h} \;\le\; C_d \;\le\; \frac{c(K - h) - c(K)}{h}.$$

As $h \to 0$ both quotients tend to $-\partial c/\partial K = e^{-r\tau}N(d_2)$ (Lemma 3), so $C_d = e^{-r\tau}N(d_2)$. The put is the same with bear spreads and $\partial p/\partial K$; the sum uses $N(d_2) + N(-d_2) = 1$. $\square$
Example: $d_1 = 0.325164$, $d_2 = -0.055836$, $e^{-r\tau} = 0.951229$; $C_d = 0.454437$, $P_d = 0.496793$, sum 0.951229. Spreads with $h = 1$: $0.450623 \le C_d \le 0.458273$.

Lemma 5 (Greeks of the digital call; differentiation of (7)).

Greek	digital call	example
$\Delta = \partial_S C_d$	$\dfrac{e^{-r\tau}\varphi(d_2)}{S\sigma\sqrt{\tau}}$	0.007640
$\Gamma = \partial_S^2 C_d$	$-\dfrac{e^{-r\tau}\varphi(d_2)\,d_1}{S^2\sigma^2\tau}$	−0.000050
vega $= \partial_\sigma C_d$	$-\dfrac{e^{-r\tau}\varphi(d_2)\,d_1}{\sigma}$	−0.323367
$\Theta = \partial_t C_d$	$re^{-r\tau}N(d_2) + e^{-r\tau}\varphi(d_2)\Big(\dfrac{d_1}{2\tau} - \dfrac{r}{\sigma\sqrt{\tau}}\Big)$	0.034599
$\rho = \partial_r C_d$	$-\tau e^{-r\tau}N(d_2) + \dfrac{e^{-r\tau}\varphi(d_2)\sqrt{\tau}}{\sigma}$	0.540037

Digital put from $P_d = e^{-r\tau} - C_d$: Δ, Γ, vega change sign; $\Theta_p = re^{-r\tau} - \Theta_c$; $\rho_p = -\tau e^{-r\tau} - \rho_c$.

Proof. Chain rule with $\varphi'(x) = -x\varphi(x)$ and the derivatives of d_2 from (3):

$$\frac{\partial d_2}{\partial S} = \frac{1}{S\sigma\sqrt{\tau}}, \quad \frac{\partial d_2}{\partial \sigma} = -\frac{d_1}{\sigma}, \quad \frac{\partial d_2}{\partial r} = \frac{\sqrt{\tau}}{\sigma}, \quad \frac{\partial d_2}{\partial \tau} = \frac{r - \sigma^2/2}{\sigma\sqrt{\tau}} - \frac{d_2}{2\tau},$$

and $\Theta = -\partial_\tau$ (calendar time, as `calc_theta` of the library). Each line was checked against a bump-and-revalue difference (agreement 10^{-6}). □

4. Q1: the code.

The price follows Lemma 4 (route: limit of bull spreads), the Greeks Lemma 5. The new classes follow the pattern of Lecture 5: the contract derives from `Contract`, the pricer from `Pricer` (Figure 1: the starting point, the library copied from the `solution` folder); each is a NEW VERSION of the lecture's class, with every change marked `# change # k` / `#Delete.` / `# replace by` / `# end change # k`.

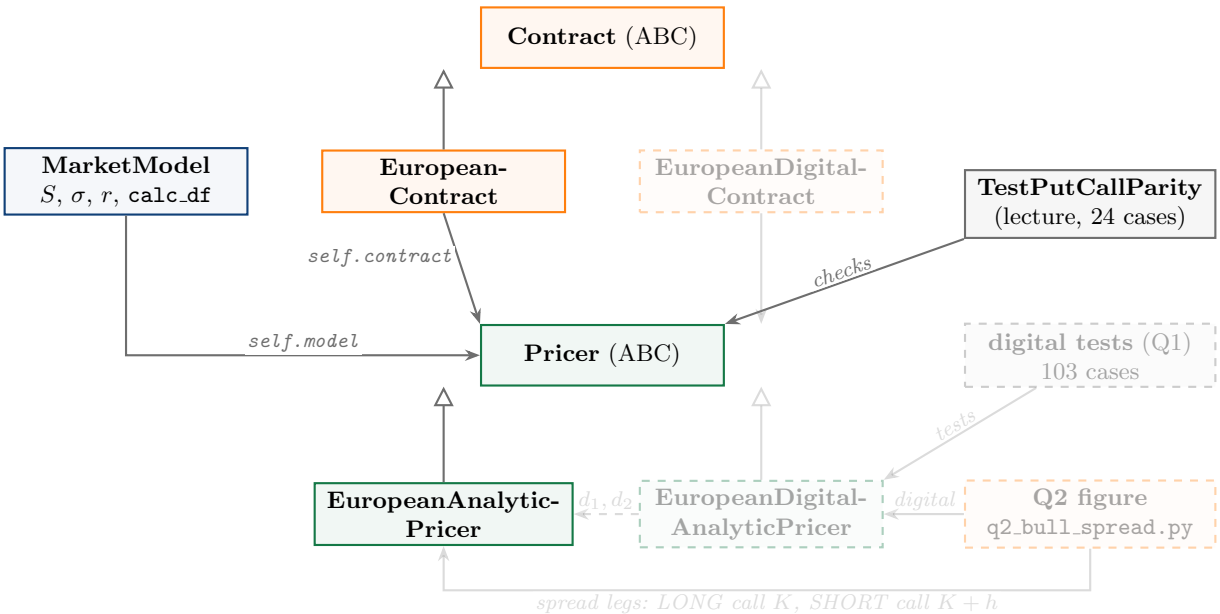


Figure 1: Blocks after copying the `solution` folder of Lecture 5 (start). Bold with NEW: created by this code; faded: still to come. Open triangle: inherits from; plain arrow: uses; dashed: reuses the static d_1, d_2.

Q1 (a): the contract. NEW VERSION of `EuropeanContract` (end of `contract.py`). Change #1: parent `Contract` (with parent `EuropeanContract` the vanilla pricer would accept a digital); change #2: the type is stored before the error is raised (the lecture's line raises `AttributeError` for a forward); changes #3–#4: the payoff of Definition 1.

```python
# ================================================================================================
# Homework 3, Q1 (NEW CODE): NEW VERSION of EuropeanContract for a digital (cash-or-nothing) option.
# The payoff is 1 if the option ends in the money, else 0 (Definition 1 of the report).
# ================================================================================================
# change # 1
#Delete.class EuropeanContract(Contract):
# replace by
class EuropeanDigitalContract(Contract):    # parent Contract, not EuropeanContract: the vanilla pricer must reject it
# end change # 1
    def __init__(self, underlying: Stock, derivative_type: PutCallFwd, long_short: LongShort, strike: float,
                 expiry: float) -> None:
        if derivative_type not in [PutCallFwd.CALL, PutCallFwd.PUT]:
            # change # 2: store the type first, so that the error message can be built (lecture code: AttributeError)
            self.derivative_type = derivative_type
            # end change # 2
            self.raise_incorrect_derivative_type_error()
        super().__init__(underlying, derivative_type, long_short, strike, expiry)

    def get_timeline(self) -> list[float]:
        return [round(self.expiry, self.timeline_digits)]

    def payoff(self, spot: dict[float, float]) -> float:
        t = self.get_timeline()[0]
        if t not in spot.keys():
            self.raise_missing_spot_error(list(spot.keys()))
        if self.derivative_type == PutCallFwd.CALL:
            # change # 3
            #Delete.    return self.direction * max(spot[t] - self.strike, 0)
            # replace by
            return self.direction if spot[t] > self.strike else 0.0     # direction * 1{S_T > K}
            # end change # 3
        elif self.derivative_type == PutCallFwd.PUT:
            # change # 4
            #Delete.    return self.direction * max(self.strike - spot[t], 0)
            # replace by
            return self.direction if spot[t] < self.strike else 0.0     # direction * 1{S_T < K}
            # end change # 4
        else:
            self.raise_incorrect_derivative_type_error()
```

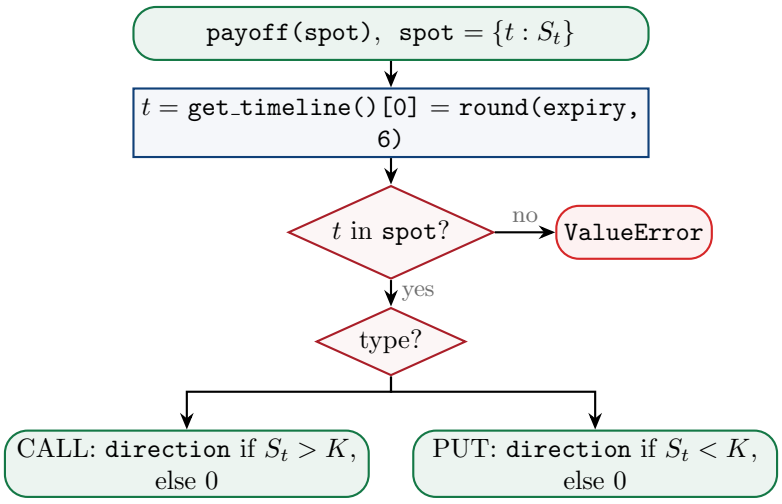


Figure 2: Flowchart of `EuropeanDigitalContract.payoff`.

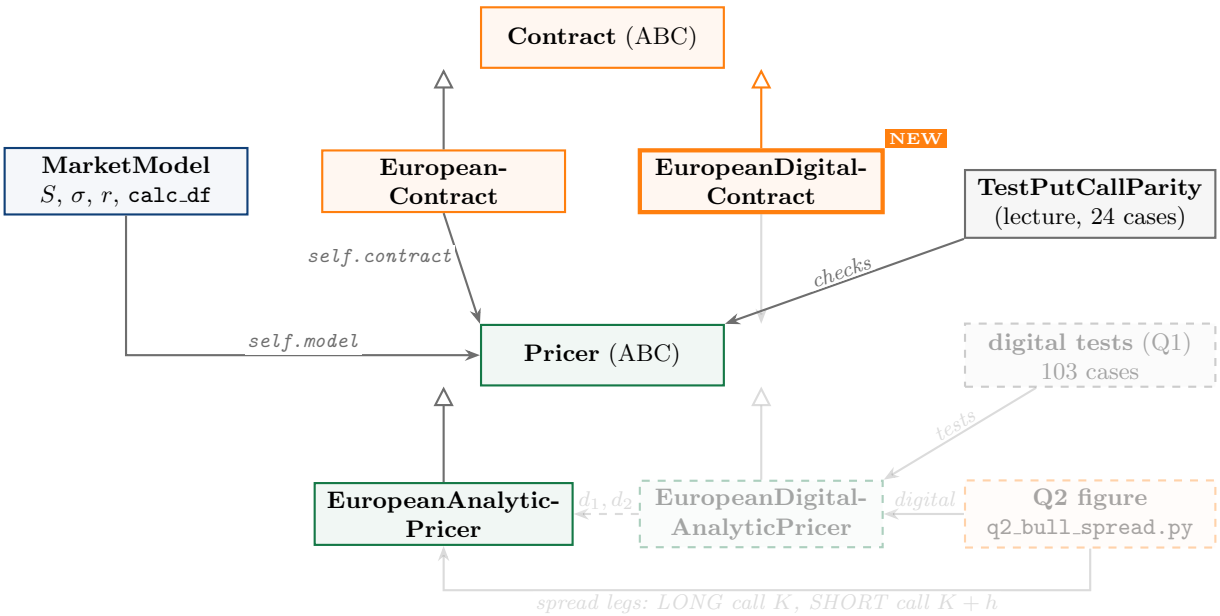


Figure 3: Blocks after Q1 (a), `EuropeanDigitalContract`. Bold with NEW: created by this code; faded: still to come. Open triangle: inherits from; plain arrow: uses; dashed: reuses the static d_1, d_2.

Q1 (b): the pricer. NEW VERSION of `EuropeanAnalyticPricer` (end of `pricer.py`). Change #1: parent `Pricer`; change #2: the type check asks for `EuropeanDigitalContract`; changes #3–#8: the fair value (Lemma 4) and the five Greeks (Lemma 5); the put uses $P_d = e^{-r\tau} - C_d$. d_1, d_2 are the static methods of the lecture's `EuropeanAnalyticPricer`.

```
# ================================================================================================
# Homework 3, Q1 (NEW CODE): NEW VERSION of EuropeanAnalyticPricer for EuropeanDigitalContract.
# Fair value (Lemma 4 of the report):  call  DF * N(d2),   put  DF * N(-d2) = DF - call   (DF = e^{-r tau})
# Greeks (Lemma 5): derivatives of DF * N(d2); the put follows from put = DF - call.
# d1, d2 are the static methods of EuropeanAnalyticPricer (reused, not copied).
# ================================================================================================
# change # 1
#Delete.class EuropeanAnalyticPricer(Pricer):
# replace by
class EuropeanDigitalAnalyticPricer(Pricer):
# end change # 1
    # change # 2
    #Delete.def __init__(self, contract: EuropeanContract, model: MarketModel, params: Params) -> None:
    #Delete.    if not isinstance(contract, EuropeanContract):
    #Delete.        raise TypeError(f'Contract must be of type EuropeanContract but received {type(contract).__name__}')
    # replace by
    def __init__(self, contract: EuropeanDigitalContract, model: MarketModel, params: Params) -> None:
        if not isinstance(contract, EuropeanDigitalContract):
            raise TypeError(f'Contract must be of type EuropeanDigitalContract but received {type(contract).__name__}')
    # end change # 2
        super().__init__(contract, model, params)

    def calc_fair_value(self) -> float:
        direction = self.contract.direction
        strike = self.contract.strike
        expiry = self.contract.expiry
        time_to_expiry = expiry - self.valuation_time
        spot = self.model.spot
        vol = self.model.vol
        rate = self.model.risk_free_rate
        df = self.model.calc_df(time_to_expiry)
        # change # 3
        #Delete.d1 = EuropeanAnalyticPricer.calc_d1(spot / strike, vol, rate, time_to_expiry)
        d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        #Delete.if self.contract.derivative_type == PutCallFwd.CALL:
        #Delete.    return direction * (spot * norm.cdf(d1) - strike * df * norm.cdf(d2))
        #Delete.elif self.contract.derivative_type == PutCallFwd.PUT:
        #Delete.    return direction * (strike * df * norm.cdf(-d2) - spot * norm.cdf(-d1))
        # replace by
        if self.contract.derivative_type == PutCallFwd.CALL:
            return direction * df * norm.cdf(d2)
        elif self.contract.derivative_type == PutCallFwd.PUT:
            return direction * df * norm.cdf(-d2)
        # end change # 3
        else:
            self.contract.raise_incorrect_derivative_type_error()

    def calc_delta(self) -> float:
        greek = 0.0
        spot = self.model.spot
        strike = self.contract.strike
```

```
        expiry = self.contract.expiry
        time_to_expiry = expiry - self.valuation_time
        vol = self.model.vol
        rate = self.model.risk_free_rate
        # change # 4
        #Delete.d1 = EuropeanAnalyticPricer.calc_d1(spot / strike, vol, rate, time_to_expiry)
        #Delete.if self.contract.derivative_type == PutCallFwd.CALL:
        #Delete.    greek = norm.cdf(d1)
        #Delete.elif self.contract.derivative_type == PutCallFwd.PUT:
        #Delete.    greek = -norm.cdf(-d1)
        # replace by
        df = self.model.calc_df(time_to_expiry)
        d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        call_greek = df * norm.pdf(d2) / (spot * vol * np.sqrt(time_to_expiry))
        if self.contract.derivative_type == PutCallFwd.CALL:
            greek = call_greek
        elif self.contract.derivative_type == PutCallFwd.PUT:
            greek = -call_greek                                  # put = DF - call
        # end change # 4
        else:
            self.contract.raise_incorrect_derivative_type_error()
        return self.contract.direction * greek

    def calc_gamma(self) -> float:
        greek = 0.0
        spot = self.model.spot
        strike = self.contract.strike
        expiry = self.contract.expiry
        time_to_expiry = expiry - self.valuation_time
        vol = self.model.vol
        rate = self.model.risk_free_rate
        # change # 5
        #Delete.d1 = EuropeanAnalyticPricer.calc_d1(spot / strike, vol, rate, time_to_expiry)
        #Delete.if self.contract.derivative_type in (PutCallFwd.CALL, PutCallFwd.PUT):
        #Delete.    greek = norm.pdf(d1) / (spot * vol * np.sqrt(time_to_expiry))
        # replace by
        df = self.model.calc_df(time_to_expiry)
        d1 = EuropeanAnalyticPricer.calc_d1(spot / strike, vol, rate, time_to_expiry)
        d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        call_greek = -df * norm.pdf(d2) * d1 / (spot**2 * vol**2 * time_to_expiry)
        if self.contract.derivative_type == PutCallFwd.CALL:
            greek = call_greek
        elif self.contract.derivative_type == PutCallFwd.PUT:
            greek = -call_greek
        # end change # 5
        else:
            self.contract.raise_incorrect_derivative_type_error()
        return self.contract.direction * greek

    def calc_vega(self) -> float:
        greek = 0.0
        spot = self.model.spot
        strike = self.contract.strike
        expiry = self.contract.expiry
        time_to_expiry = expiry - self.valuation_time
        vol = self.model.vol
        rate = self.model.risk_free_rate
        df = self.model.calc_df(time_to_expiry)
        # change # 6
        #Delete.d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        #Delete.if self.contract.derivative_type in (PutCallFwd.CALL, PutCallFwd.PUT):
        #Delete.    greek = strike * df * norm.pdf(d2) * np.sqrt(time_to_expiry)
        # replace by
        d1 = EuropeanAnalyticPricer.calc_d1(spot / strike, vol, rate, time_to_expiry)
        d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        call_greek = -df * norm.pdf(d2) * d1 / vol
        if self.contract.derivative_type == PutCallFwd.CALL:
            greek = call_greek
        elif self.contract.derivative_type == PutCallFwd.PUT:
            greek = -call_greek
        # end change # 6
        else:
            self.contract.raise_incorrect_derivative_type_error()
        return self.contract.direction * greek

    def calc_theta(self) -> float:
        greek = 0.0
        strike = self.contract.strike
        spot = self.model.spot
        expiry = self.contract.expiry
        time_to_expiry = expiry - self.valuation_time
        vol = self.model.vol
        rate = self.model.risk_free_rate
        df = self.model.calc_df(time_to_expiry)
        d1 = EuropeanAnalyticPricer.calc_d1(spot / strike, vol, rate, time_to_expiry)
        d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        # change # 7
        #Delete.if self.contract.derivative_type == PutCallFwd.CALL:
        #Delete.    greek = -1.0 * ((spot * norm.pdf(d1) * vol) / (2 * np.sqrt(time_to_expiry))
        #Delete.                    + rate * strike * df * norm.cdf(d2))
        #Delete.elif self.contract.derivative_type == PutCallFwd.PUT:
        #Delete.    greek = -1.0 * ((spot * norm.pdf(d1) * vol) / (2 * np.sqrt(time_to_expiry))
        #Delete.                    - rate * strike * df * norm.cdf(-d2))
```

```
        # replace by
        call_greek = (rate * df * norm.cdf(d2)
                      + df * norm.pdf(d2) * (d1 / (2 * time_to_expiry) - rate / (vol * np.sqrt(time_to_expiry))))
        if self.contract.derivative_type == PutCallFwd.CALL:
            greek = call_greek
        elif self.contract.derivative_type == PutCallFwd.PUT:
            greek = rate * df - call_greek                      # theta of DF = e^{-r(T-t)} is r * DF
        # end change # 7
        else:
            self.contract.raise_incorrect_derivative_type_error()
        return self.contract.direction * greek

    def calc_rho(self) -> float:
        greek = 0.0
        strike = self.contract.strike
        expiry = self.contract.expiry
        time_to_expiry = expiry - self.valuation_time
        spot = self.model.spot
        vol = self.model.vol
        rate = self.model.risk_free_rate
        df = self.model.calc_df(time_to_expiry)
        d2 = EuropeanAnalyticPricer.calc_d2(spot / strike, vol, rate, time_to_expiry)
        # change # 8
        #Delete.if self.contract.derivative_type == PutCallFwd.CALL:
        #Delete.    greek = strike * time_to_expiry * df * norm.cdf(d2)
        #Delete.elif self.contract.derivative_type == PutCallFwd.PUT:
        #Delete.    greek = -strike * time_to_expiry * df * norm.cdf(-d2)
        # replace by
        call_greek = -time_to_expiry * df * norm.cdf(d2) + df * norm.pdf(d2) * np.sqrt(time_to_expiry) / vol
        if self.contract.derivative_type == PutCallFwd.CALL:
            greek = call_greek
        elif self.contract.derivative_type == PutCallFwd.PUT:
            greek = -time_to_expiry * df - call_greek            # rho of DF = e^{-r tau} is -tau * DF
        # end change # 8
        else:
            self.contract.raise_incorrect_derivative_type_error()
        return self.contract.direction * greek
```

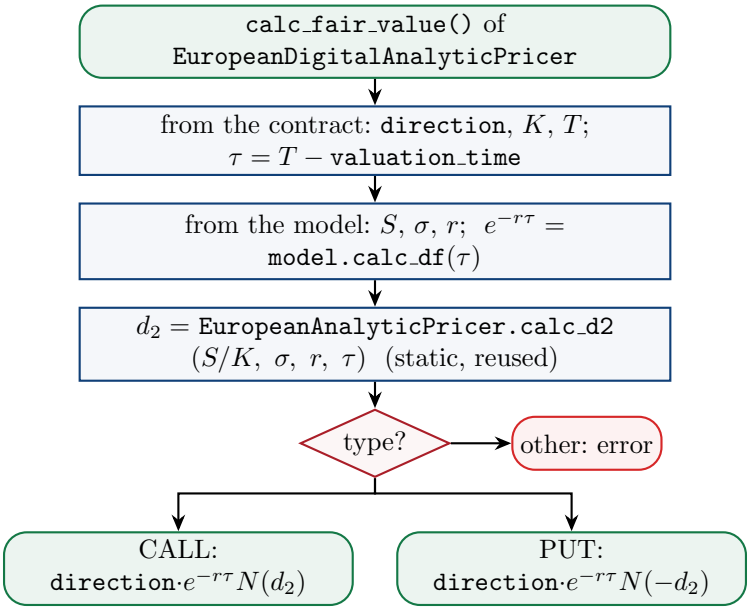


Figure 4: Flowchart of `calc_fair_value` of `EuropeanDigitalAnalyticPricer` (same skeleton as `EuropeanAnalyticPricer`, Lecture 5). The five Greeks follow the same steps with the formulas of Lemma 5.

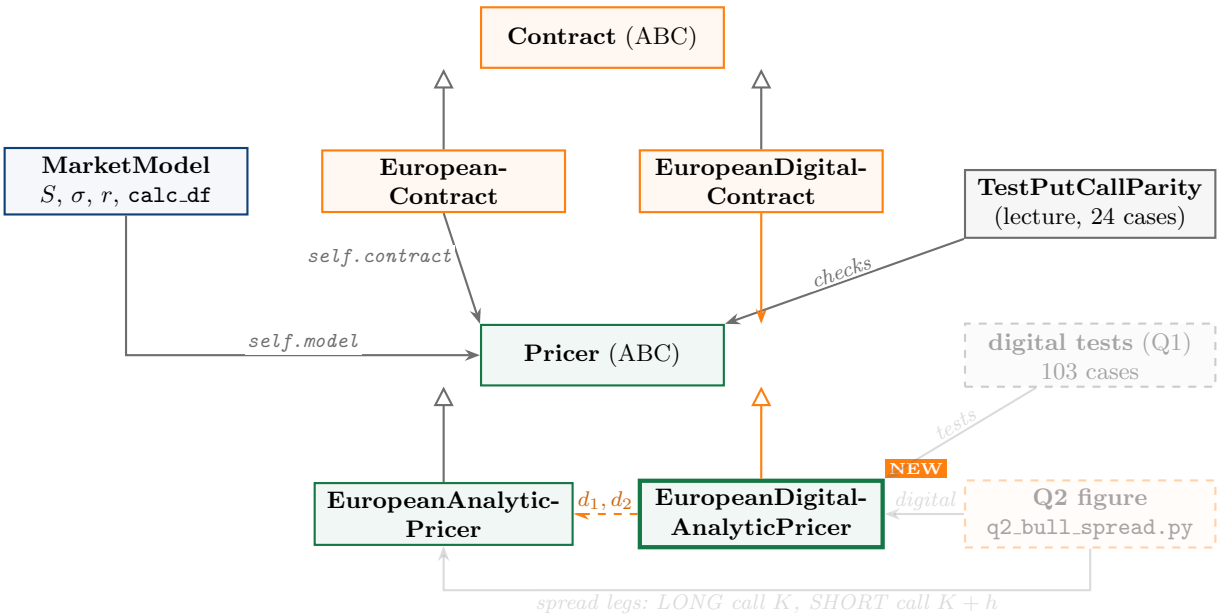


Figure 5: Blocks after Q1 (b), `EuropeanDigitalAnalyticPricer`. Bold with NEW: created by this code; faded: still to come. Open triangle: inherits from; plain arrow: uses; dashed: reuses the static d_1, d_2.

Q1 (c): test of the new classes. File `q1_digital_demo.py`: payoffs, fair values and parity (Lemma 4), the bull-spread bounds of the proof of Lemma 4 with the lecture's vanilla pricer, and the Greeks against bump and revaluation.

```
# Homework 3, Q1: the digital option in the library (contract + analytic pricer), running example.
# Run from this folder with the course venv:  python q1_digital_demo.py   (printed output: q1_digital_demo.out)
import sys
from pathlib import Path

here = Path(__file__).resolve().parent
root = next((p for p in [here, *here.parents] if (p / 'src' / 'enums.py').exists()), here)
sys.path[:0] = [str(here), str(root)]               # this folder (library) and the repository (src.enums)

from pricer import *                                # library of Lecture 5 + the NEW classes (Homework 3)

# running example: the market of the library
model = MarketModel(Stock.BLUECHIP_BANK)
strike, expiry = 130.0, 1.0
print(f'market: S = {model.spot}, vol = {model.vol}, r = {model.risk_free_rate};  K = {strike}, T = {expiry}')

# 1. payoffs of the new contract (Definition 1)
long_call = EuropeanDigitalContract(Stock.BLUECHIP_BANK, PutCallFwd.CALL, LongShort.LONG, strike, expiry)
short_put = EuropeanDigitalContract(Stock.BLUECHIP_BANK, PutCallFwd.PUT, LongShort.SHORT, strike, expiry)
for s_T in [129.0, 131.0]:
    print(f'S_T = {s_T}: long digital call pays {long_call.payoff({expiry: s_T})}, '
          f'short digital put pays {short_put.payoff({expiry: s_T})}')

# 2. fair values (Lemma 4) and parity C_d + P_d = DF
pricers = {cp: EuropeanDigitalAnalyticPricer(
               EuropeanDigitalContract(Stock.BLUECHIP_BANK, cp, LongShort.LONG, strike, expiry), model, Params())
           for cp in (PutCallFwd.CALL, PutCallFwd.PUT)}
call, put = (pricers[cp].calc_fair_value() for cp in (PutCallFwd.CALL, PutCallFwd.PUT))
df = model.calc_df(expiry)
d2 = EuropeanAnalyticPricer.calc_d2(model.spot / strike, model.vol, model.risk_free_rate, expiry)
print(f'd2 = {d2:.6f}, DF = {df:.6f}')
print(f'digital call = {call:.6f}, digital put = {put:.6f}, call + put = {call + put:.6f} (DF = {df:.6f})')


# 3. bull spreads with the lecture's vanilla pricer (proof of Lemma 4)
def vanilla_call(k: float) -> float:
    contract = EuropeanContract(Stock.BLUECHIP_BANK, PutCallFwd.CALL, LongShort.LONG, k, expiry)
    return EuropeanAnalyticPricer(contract, model, Params()).calc_fair_value()


for h in [10.0, 1.0, 0.1, 0.01]:
    lower = (vanilla_call(strike) - vanilla_call(strike + h)) / h
    upper = (vanilla_call(strike - h) - vanilla_call(strike)) / h
    print(f'h = {h:5.2f}: {lower:.6f} <= C_d = {call:.6f} <= {upper:.6f}   {lower <= call <= upper}')
```

```python
# 4. Greeks (Lemma 5) against bump and revaluation
print(f"{'':6s}{'call':>11s}{'put':>11s}{'call bump':>11s}")
pc = pricers[PutCallFwd.CALL]
hs, h = 0.01, 1e-4


def bumped(attribute: str, step: float) -> float:
    owner = pc if attribute == 'valuation_time' else model
    base = getattr(owner, attribute)
    setattr(owner, attribute, base + step)
    value = pc.calc_fair_value()
    setattr(owner, attribute, base)
    return value


bumps = {'delta': (bumped('spot', hs) - bumped('spot', -hs)) / (2 * hs),
         'gamma': (bumped('spot', hs) - 2 * call + bumped('spot', -hs)) / hs**2,
         'vega': (bumped('vol', h) - bumped('vol', -h)) / (2 * h),
         'theta': (bumped('valuation_time', h) - bumped('valuation_time', -h)) / (2 * h),
         'rho': (bumped('risk_free_rate', h) - bumped('risk_free_rate', -h)) / (2 * h)}
for g, b in bumps.items():
    gc = getattr(pricers[PutCallFwd.CALL], f'calc_{g}')()
    gp = getattr(pricers[PutCallFwd.PUT], f'calc_{g}')()
    print(f'{g:6s}{gc:11.6f}{gp:11.6f}{b:11.6f}')
```

```
market: S = 130.17, vol = 0.381, r = 0.05;  K = 130.0, T = 1.0
S_T = 129.0: long digital call pays 0.0, short digital put pays -1.0
S_T = 131.0: long digital call pays 1.0, short digital put pays 0.0
d2 = -0.055836, DF = 0.951229
digital call = 0.454437, digital put = 0.496793, call + put = 0.951229 (DF = 0.951229)
h = 10.00: 0.417385 <= C_d = 0.454437 <= 0.493716   True
h =  1.00: 0.450623 <= C_d = 0.454437 <= 0.458273   True
h =  0.10: 0.454054 <= C_d = 0.454437 <= 0.454819   True
h =  0.01: 0.454398 <= C_d = 0.454437 <= 0.454475   True
              call        put  call bump
delta     0.007640  -0.007640   0.007640
gamma    -0.000050   0.000050  -0.000050
vega     -0.323367   0.323367  -0.323367
theta     0.034599   0.012962   0.034599
rho       0.540037  -1.491267   0.540037
```

Reading: $C_d = 0.454437$ and $P_d = 0.496793$ add up to $e^{-0.05} = 0.951229$ (Lemma 4); every bull-spread pair brackets C_d and the bracket shrinks with h (proof of Lemma 4); each Greek equals its bump (Lemma 5); the put's Greeks are those of $e^{-r\tau} - C_d$.

Q1 (d): the unit tests. End of `test_pricer.py`, after the professor's `TestPutCallParity`. The fair value is tested in `TestDigitalFairValue` on the grid of the lecture (2 stocks $\times$ 3 values of K/S $\times$ 4 expiries) and in `test_digital_value_by_hand`; the payoff, the type checks and the Greeks have their own tests.

```python
# ================================================================================================
# Homework 3, Q1 (NEW CODE): unit tests of the digital option.
# Fair value (asked in the assignment) + payoff, type checks and Greeks.
# ================================================================================================
def digital_pricer(model: MarketModel, call_put: PutCallFwd, long_short: LongShort, strike: float,
                   expiry: float) -> EuropeanDigitalAnalyticPricer:
    contract = EuropeanDigitalContract(model.underlying, call_put, long_short, strike, expiry)
    return EuropeanDigitalAnalyticPricer(contract, model, Params())


def call_price(model: MarketModel, strike: float, expiry: float) -> float:
    """Long vanilla call of the lecture (EuropeanAnalyticPricer)."""
    contract = EuropeanContract(model.underlying, PutCallFwd.CALL, LongShort.LONG, strike, expiry)
    return EuropeanAnalyticPricer(contract, model, Params()).calc_fair_value()


@pytest.mark.parametrize('underlying', [Stock.BLUECHIP_BANK, Stock.TIPTOP_SOLUTIONS])
@pytest.mark.parametrize('strike_over_spot', np.arange(0.5, 2.0, 0.5))
@pytest.mark.parametrize('expiry', np.arange(0.5, 2.5, 0.5))
class TestDigitalFairValue:
    def test_digital_parity(self, underlying, strike_over_spot, expiry):
        # long digital call + long digital put pay 1 at T, worth DF = e^{-rT} today (Lemma 4)
        model = MarketModel(underlying)
        strike = strike_over_spot * model.spot
```

```
        call = digital_pricer(model, PutCallFwd.CALL, LongShort.LONG, strike, expiry).calc_fair_value()
        put = digital_pricer(model, PutCallFwd.PUT, LongShort.LONG, strike, expiry).calc_fair_value()
        assert call + put == pytest.approx(model.calc_df(expiry), abs=1e-12)

    def test_bull_spread_bounds(self, underlying, strike_over_spot, expiry):
        # (c(K) - c(K+h))/h <= C_d <= (c(K-h) - c(K))/h for every h > 0 (proof of Lemma 4); h = 1% of K
        model = MarketModel(underlying)
        strike = strike_over_spot * model.spot
        h = 0.01 * strike
        digital = digital_pricer(model, PutCallFwd.CALL, LongShort.LONG, strike, expiry).calc_fair_value()
        lower = (call_price(model, strike, expiry) - call_price(model, strike + h, expiry)) / h
        upper = (call_price(model, strike - h, expiry) - call_price(model, strike, expiry)) / h
        assert lower <= digital <= upper

    def test_bull_spread_limit(self, underlying, strike_over_spot, expiry):
        # a narrow scaled bull spread (width h = 0.01% of K) has the price of the digital call (Lemma 4)
        model = MarketModel(underlying)
        strike = strike_over_spot * model.spot
        h = 1e-4 * strike
        digital = digital_pricer(model, PutCallFwd.CALL, LongShort.LONG, strike, expiry).calc_fair_value()
        spread = (call_price(model, strike - h / 2, expiry) - call_price(model, strike + h / 2, expiry)) / h
        assert spread == pytest.approx(digital, abs=1e-6)

    def test_short_is_minus_long(self, underlying, strike_over_spot, expiry):
        model = MarketModel(underlying)
        strike = strike_over_spot * model.spot
        for call_put in (PutCallFwd.CALL, PutCallFwd.PUT):
            long = digital_pricer(model, call_put, LongShort.LONG, strike, expiry).calc_fair_value()
            short = digital_pricer(model, call_put, LongShort.SHORT, strike, expiry).calc_fair_value()
            assert short == pytest.approx(-long, abs=1e-15)


def test_digital_value_by_hand():
    # BLUECHIP_BANK (S = 130.17, vol = 0.381, r = 0.05), K = 130, T = 1:
    # d2 = -0.055836, DF = 0.951229, C_d = DF * N(d2) = 0.454437, P_d = DF * N(-d2) = 0.496793
    model = MarketModel(Stock.BLUECHIP_BANK)
    call = digital_pricer(model, PutCallFwd.CALL, LongShort.LONG, 130.0, 1.0).calc_fair_value()
    put = digital_pricer(model, PutCallFwd.PUT, LongShort.LONG, 130.0, 1.0).calc_fair_value()
    assert call == pytest.approx(0.454437, abs=1e-6)
    assert put == pytest.approx(0.496793, abs=1e-6)


def test_digital_payoff():
    long_call = EuropeanDigitalContract(Stock.BLUECHIP_BANK, PutCallFwd.CALL, LongShort.LONG, 130.0, 1.0)
    short_put = EuropeanDigitalContract(Stock.BLUECHIP_BANK, PutCallFwd.PUT, LongShort.SHORT, 130.0, 1.0)
    assert long_call.payoff({1.0: 131.0}) == 1.0
    assert long_call.payoff({1.0: 129.0}) == 0.0
    assert short_put.payoff({1.0: 129.0}) == -1.0
    assert short_put.payoff({1.0: 131.0}) == 0.0
    with pytest.raises(ValueError):
        long_call.payoff({0.5: 131.0})                # no spot at the expiry


def test_digital_type_checks():
    model = MarketModel(Stock.BLUECHIP_BANK)
    vanilla = EuropeanContract(Stock.BLUECHIP_BANK, PutCallFwd.CALL, LongShort.LONG, 130.0, 1.0)
    digital = EuropeanDigitalContract(Stock.BLUECHIP_BANK, PutCallFwd.CALL, LongShort.LONG, 130.0, 1.0)
    with pytest.raises(ValueError):
        EuropeanDigitalContract(Stock.BLUECHIP_BANK, PutCallFwd.FWD, LongShort.LONG, 130.0, 1.0)
    with pytest.raises(TypeError):
        EuropeanDigitalAnalyticPricer(vanilla, model, Params())      # a vanilla is not a digital
    with pytest.raises(TypeError):
        EuropeanAnalyticPricer(digital, model, Params())             # the vanilla pricer rejects a digital


def bumped_value(pricer: EuropeanDigitalAnalyticPricer, attribute: str, h: float) -> float:
    """Fair value after adding h to the model's spot / vol / risk_free_rate, or to the valuation time."""
    owner = pricer if attribute == 'valuation_time' else pricer.model
    base = getattr(owner, attribute)
    setattr(owner, attribute, base + h)
    value = pricer.calc_fair_value()
    setattr(owner, attribute, base)
    return value


@pytest.mark.parametrize('call_put', [PutCallFwd.CALL, PutCallFwd.PUT])
@pytest.mark.parametrize('long_short', [LongShort.LONG, LongShort.SHORT])
def test_digital_greeks_bump(call_put, long_short):
    # analytic Greeks (Lemma 5) = central differences of the fair value
    pricer = digital_pricer(MarketModel(Stock.BLUECHIP_BANK), call_put, long_short, 130.0, 1.0)
    fv = pricer.calc_fair_value()
```

```
    hs, h = 0.01, 1e-4
    bump = {
        'delta': (bumped_value(pricer, 'spot', hs) - bumped_value(pricer, 'spot', -hs)) / (2 * hs),
        'gamma': (bumped_value(pricer, 'spot', hs) - 2 * fv + bumped_value(pricer, 'spot', -hs)) / hs**2,
        'vega': (bumped_value(pricer, 'vol', h) - bumped_value(pricer, 'vol', -h)) / (2 * h),
        'theta': (bumped_value(pricer, 'valuation_time', h) - bumped_value(pricer, 'valuation_time', -h)) / (2 * h),
        'rho': (bumped_value(pricer, 'risk_free_rate', h) - bumped_value(pricer, 'risk_free_rate', -h)) / (2 * h)}
    analytic = {'delta': pricer.calc_delta(), 'gamma': pricer.calc_gamma(), 'vega': pricer.calc_vega(),
                'theta': pricer.calc_theta(), 'rho': pricer.calc_rho()}
    assert analytic == pytest.approx(bump, rel=1e-4, abs=1e-8)
```

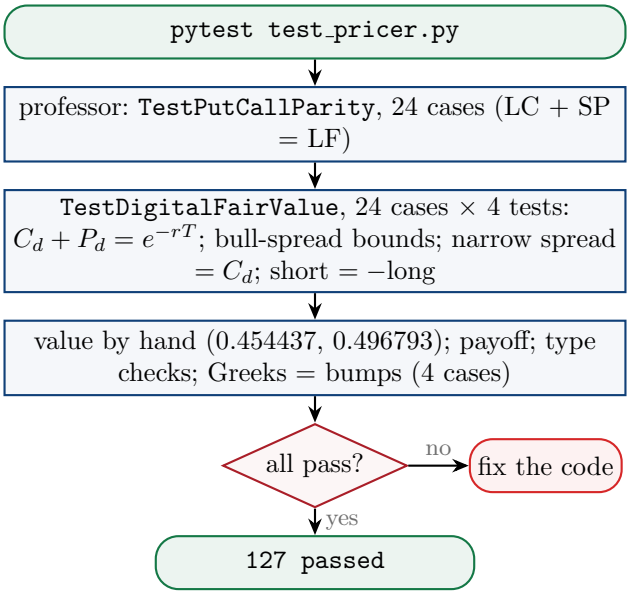


Figure 6: Flowchart of the unit tests.

Run from the repository folder, and output:

```
python -m pytest -q homeworks/54932fw/3_HW_LECTURE_5/test_pricer.py
```

```
........................................................................ [ 56%]
.....................................................                    [100%]
127 passed in 0.85s
```

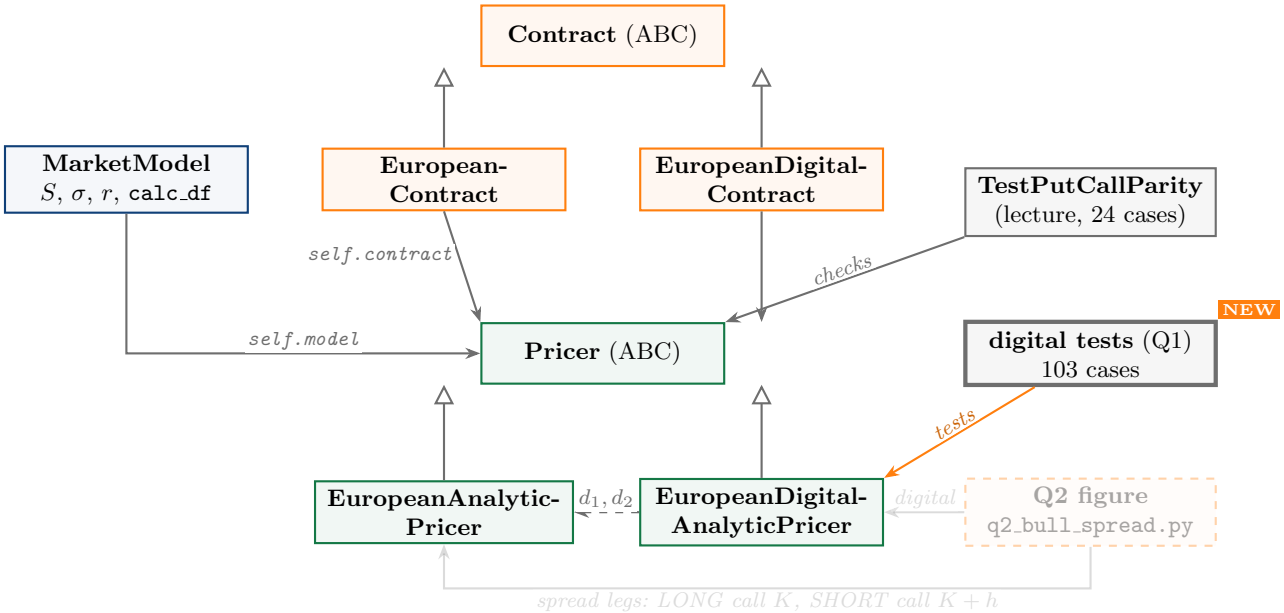


Figure 7: Blocks after Q1 (d), the unit tests. Bold with NEW: created by this code; faded: still to come. Open triangle: inherits from; plain arrow: uses; dashed: reuses the static d_1, d_2.

Q1. `EuropeanDigitalContract` pays `direction` $\cdot \mathbf{1}\{S_T > K\}$ (call) or `direction` $\cdot \mathbf{1}\{S_T < K\}$ (put); `EuropeanDigitalAnalyticPricer` returns $C_d = e^{-r\tau}N(d_2)$, $P_d = e^{-r\tau}N(-d_2)$ and the five Greeks of Lemma 5, times `direction`. All 127 unit tests pass (the professor's 24 and 103 new ones). Running example: $C_d = 0.454437$, $P_d = 0.496793$.

5. Q2: the digital call as a limit of bull call spreads.

Get familiar with bull spread options and by utilizing EuropeanDigitalAnalyticPricer and EuropeanAnalyticPricer, create a figure to visualize that a digital call option can be thought of as a limit of bull spread call options.

Idea (the proof of Lemma 4 as a picture). A bull spread of width h (Definition 2) is a LONG call with strike K and a SHORT call with strike $K+h$; it pays at most h, so we hold $1/h$ of it. Its payoff tends to $\mathbf{1}\{S_T > K\}$ as $h \to 0$, and its price $R_h = \frac{c(K)-c(K+h)}{h}$ tends to C_d (Lemma 4). The left spread $L_h = \frac{c(K-h)-c(K)}{h}$ approaches C_d from above. In the library a spread is two `EuropeanAnalyticPricer`s, the second with `LongShort.SHORT`: its `direction` $= -1$ makes the sum of the two fair values the price of the spread. The digital is priced with `EuropeanDigitalAnalyticPricer` (Q1). The figure has three panels: (a) payoffs at T, (b) prices today against the spot, (c) the error against h.

Q2 (a): the code (file `q2_bull_spread.py`; the plots use `my_plotter` of Homework 1, file `plotter.py`).

```
# Homework 3, Q2: a digital call is the limit of (scaled) bull call spreads.
# Uses EuropeanAnalyticPricer (lecture) for the spreads and EuropeanDigitalAnalyticPricer (Q1) for the digital;
# the plots are drawn with my_plotter of Homework 1 (plotter.py).
# Run from this folder with the course venv:  python q2_bull_spread.py   (output: q2_bull_spread.out, figures/)
import sys
from pathlib import Path

here = Path(__file__).resolve().parent
root = next((p for p in [here, *here.parents] if (p / 'src' / 'enums.py').exists()), here)
sys.path[:0] = [str(here), str(root)]               # this folder (library) and the repository (src.enums)

import matplotlib
matplotlib.use('Agg')                               # files only (no window)
import matplotlib.pyplot as plt
import numpy as np
from pricer import *                                # library of Lecture 5 + the classes of Q1
from plotter import my_plotter                      # Homework 1 (HW 2/1)

stock, strike, expiry = Stock.BLUECHIP_BANK, 130.0, 1.0
model = MarketModel(stock)


def bull_spread(k_low: float, k_high: float) -> list[EuropeanAnalyticPricer]:
    """Bull call spread (Definition 2): LONG call with strike k_low + SHORT call with strike k_high."""
    long_leg = EuropeanContract(stock, PutCallFwd.CALL, LongShort.LONG, k_low, expiry)
    short_leg = EuropeanContract(stock, PutCallFwd.CALL, LongShort.SHORT, k_high, expiry)
    return [EuropeanAnalyticPricer(long_leg, model, Params()), EuropeanAnalyticPricer(short_leg, model, Params())]


def scaled_value(legs: list[EuropeanAnalyticPricer], h: float) -> float:
    """Price of (1/h) x bull spread: the sum of the two legs (the SHORT leg is negative), divided by h."""
    return sum(leg.calc_fair_value() for leg in legs) / h


def scaled_payoff(legs: list[EuropeanAnalyticPricer], h: float, s_T: float) -> float:
    return sum(leg.contract.payoff({expiry: s_T}) for leg in legs) / h


def at_spot(pricer_value, s: float) -> float:
    """Value with the model's spot set to s (the model is shared by all pricers), then restore it."""
    base = model.spot
    model.spot = s
    value = pricer_value()
    model.spot = base
    return value


digital_contract = EuropeanDigitalContract(stock, PutCallFwd.CALL, LongShort.LONG, strike, expiry)
digital = EuropeanDigitalAnalyticPricer(digital_contract, model, Params())
c_d = digital.calc_fair_value()
print(f'market: S = {model.spot}, vol = {model.vol}, r = {model.risk_free_rate};  K = {strike}, T = {expiry}')
```

```
print(f'digital call C_d = {c_d:.6f}')

# 1. table: right spread R_h = (1/h)(K, K+h) <= C_d <= left spread L_h = (1/h)(K-h, K)  (proof of Lemma 4)
print(f"{'h':>7s}{'R_h':>11s}{'L_h':>11s}{'C_d - R_h':>12s}{'L_h - C_d':>12s}")
widths = np.array([40.0, 20.0, 10.0, 5.0, 2.0, 1.0, 0.5, 0.1, 0.01])
right = np.array([scaled_value(bull_spread(strike, strike + h), h) for h in widths])
left = np.array([scaled_value(bull_spread(strike - h, strike), h) for h in widths])
for h, r_h, l_h in zip(widths, right, left):
    print(f'{h:7.2f}{r_h:11.6f}{l_h:11.6f}{c_d - r_h:12.2e}{l_h - c_d:12.2e}')
slope = np.polyfit(np.log(widths[-4:]), np.log(c_d - right[-4:]), 1)[0]
print(f'error of R_h ~ h^{slope:.2f} for small h (first order)')

# 2. figure: (a) payoffs at T, (b) prices today against the spot, (c) price error against h
plot_widths = [20.0, 10.0, 2.0]
colors = ['tab:green', 'tab:orange', 'tab:blue', 'black']
names = [f'bull spread / h, h = {h:g}' for h in plot_widths] + ['digital call']
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

s_T = np.linspace(100.0, 160.0, 601)
payoffs = [[scaled_payoff(bull_spread(strike, strike + h), h, s) for s in s_T] for h in plot_widths]
payoffs.append([digital.contract.payoff({expiry: s}) for s in s_T])
my_plotter(s_T, payoffs, names=names, ax=axes[0],
           layout={'title': '(a) payoff at expiry', 'x_label': '$S_T$', 'y_label': 'payoff',
                   'colors': colors, 'linestyles': ['-', '-', '-', '--'], 'linewidth': 2, 'title_fontsize': 14,
                   'label_fontsize': 12, 'legend_fontsize': 10, 'title_pad': 10, 'label_pad': 5,
                   'hline': None, 'vlines': [strike], 'grid': True, 'legend_loc': 'upper left'})

spots = np.linspace(60.0, 220.0, 161)
prices = [[at_spot(lambda: scaled_value(bull_spread(strike, strike + h), h), s) for s in spots] for h in plot_widths]
prices.append([at_spot(digital.calc_fair_value, s) for s in spots])
my_plotter(spots, prices, names=names, ax=axes[1],
           layout={'title': '(b) price today', 'x_label': 'spot $S$', 'y_label': 'price',
                   'colors': colors, 'linestyles': ['-', '-', '-', '--'], 'linewidth': 2, 'title_fontsize': 14,
                   'label_fontsize': 12, 'legend_fontsize': 10, 'title_pad': 10, 'label_pad': 5,
                   'hline': None, 'vlines': [strike], 'grid': True, 'legend_loc': 'upper left'})

my_plotter(widths, [c_d - right, left - c_d], names=['$C_d - R_h$ (right spread)', '$L_h - C_d$ (left spread)'],
           ax=axes[2],
           layout={'title': '(c) error against the width h', 'x_label': 'h (log scale)',
                   'y_label': 'error (log scale)', 'colors': ['tab:blue', 'tab:red'], 'markers': ['o', 's'],
                   'linewidth': 2, 'title_fontsize': 14, 'label_fontsize': 12, 'legend_fontsize': 10,
                   'title_pad': 10, 'label_pad': 5, 'hline': None, 'logx': True, 'logy': True, 'grid': True,
                   'legend_loc': 'upper left'})
fig.suptitle(f'Digital call = limit of bull call spreads  (BLUECHIP_BANK, K = {strike:g}, T = {expiry:g},'
             f' C_d = {c_d:.4f})', fontsize=15)
fig.tight_layout()
(here / 'figures').mkdir(exist_ok=True)
fig.savefig(here / 'figures' / 'q2_digital_vs_bull_spreads.pdf')
fig.savefig(here / 'figures' / 'q2_digital_vs_bull_spreads.png', dpi=110)
print('figure saved: figures/q2_digital_vs_bull_spreads.pdf (and .png)')
```

```
market: S = 130.17, vol = 0.381, r = 0.05;  K = 130.0, T = 1.0
digital call C_d = 0.454437
      h        R_h        L_h   C_d - R_h   L_h - C_d
  40.00   0.322478   0.617161    1.32e-01    1.63e-01
  20.00   0.382947   0.534585    7.15e-02    8.01e-02
  10.00   0.417385   0.493716    3.71e-02    3.93e-02
   5.00   0.435603   0.473831    1.88e-02    1.94e-02
   2.00   0.446832   0.462131    7.60e-03    7.69e-03
   1.00   0.450623   0.458273    3.81e-03    3.84e-03
   0.50   0.452527   0.456352    1.91e-03    1.92e-03
   0.10   0.454054   0.454819    3.82e-04    3.83e-04
   0.01   0.454398   0.454475    3.82e-05    3.83e-05
error of R_h ~ h^1.00 for small h (first order)
figure saved: figures/q2_digital_vs_bull_spreads.pdf (and .png)
```

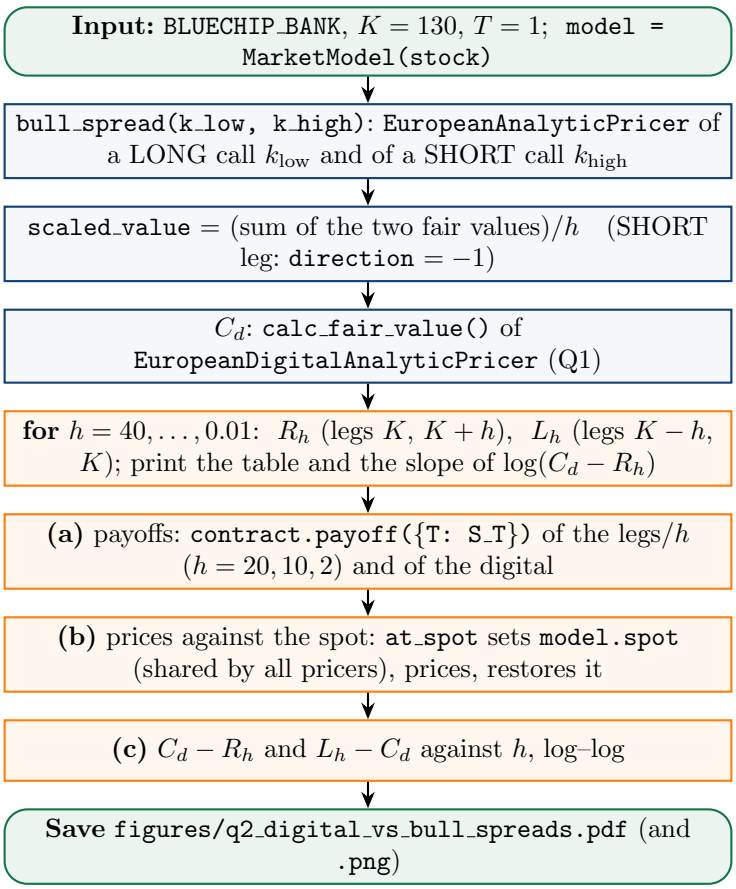


Figure 8: Flowchart of `q2_bull_spread.py`.

Q2 (b): the figure (drawn here with the same numbers as the figure the script saves in `figures/`).

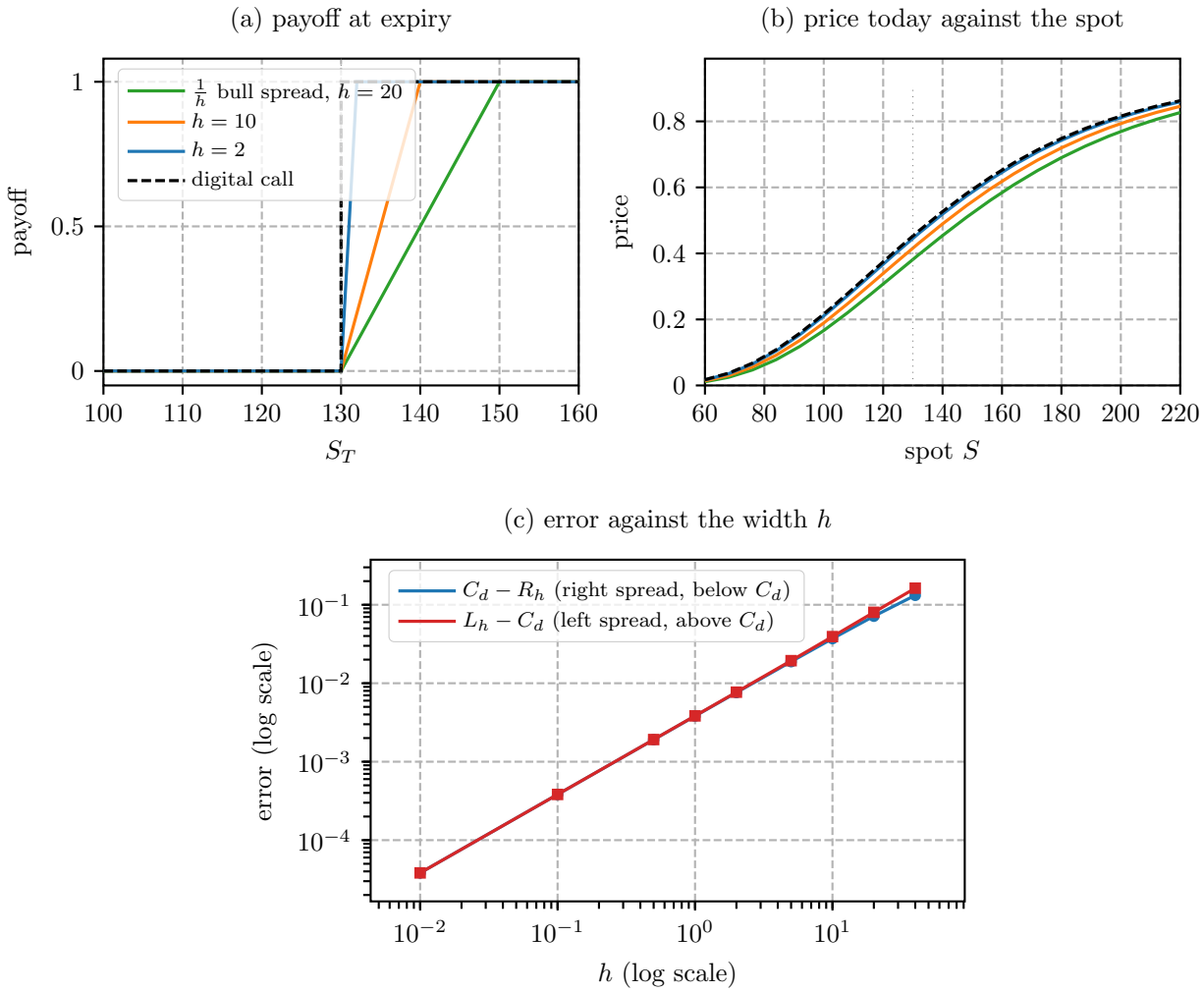


Figure 9: The digital call as a limit of bull call spreads (`BLUECHIP_BANK`, $K = 130$, $T = 1$, $C_d = 0.4544$). (a) The payoff of $\frac{1}{h}$ bull spread differs from the digital only on $(K, K + h)$. (b) For every spot the prices of the scaled spreads ($h = 20, 10, 2$, `EuropeanAnalyticPricer`) approach the digital price (dashed, `EuropeanDigitalAnalyticPricer`). (c) Both errors go to 0 like h.

Reading: in (a) the ramp of width h becomes the step of the digital; in (b) the curve for $h = 2$ is already on top of the digital curve; in (c) the right spreads stay below C_d and the left spreads above it (the sandwich of the proof of Lemma 4), and both errors decrease like h (slope 1.00 in the output; e.g. $h = 0.01$: 3.8×10^{-5}).

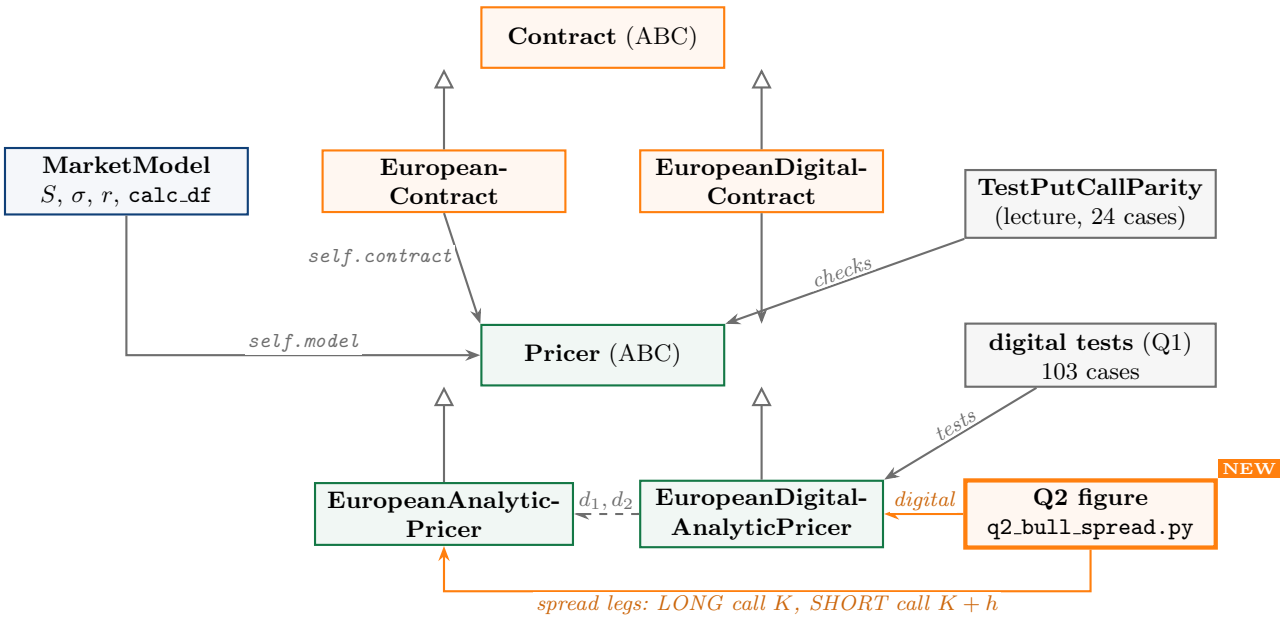


Figure 10: Blocks after Q2, `q2_bull_spread.py`: all blocks of the homework. Bold with NEW: created by this code; faded: still to come. Open triangle: inherits from; plain arrow: uses; dashed: reuses the static d_1, d_2.

Q2. A digital call is the limit of $\frac{1}{h}$ bull call spreads (LONG call K, SHORT call $K+h$): the payoffs tend to $\mathbf{1}\{S_T > K\}$, and the prices, computed with `EuropeanAnalyticPricer`, tend to the price of `EuropeanDigitalAnalyticPricer`, $C_d = e^{-r\tau}N(d_2)$, for every spot (Figure 9). Running example: $R_{0.01} = 0.454398 < C_d = 0.454437 < L_{0.01} = 0.454475$.

Comment (to keep in mind).

- *Other routes to Lemma 4.* The formula is also the risk-neutral value $e^{-r\tau}\,\mathbb{Q}(S_T > K)$, with S_T log-normal under $\mathbb{Q}$; continuous-time risk-neutral pricing is not presented in the lectures (only its tree version, Lecture 4), see [5]. And $f = e^{-r\tau}N(d_2)$ satisfies the Black–Scholes PDE of Theorem 2 with $f \to \mathbf{1}\{S > K\}$ as $\tau \to 0$ (checked numerically: residual $\approx 10^{-10}$).
- *One-sided and centred spreads.* R_h and L_h have error of order h (panel (c)); their average, the centred spread $\frac{c(K-h/2)-c(K+h/2)}{h}$, has error of order h^2, which is why the unit test `test_bull_spread_limit` uses it with a small tolerance.
- *Lecture code.* `EuropeanContract(..., FWD, ...)` raises `AttributeError` instead of the intended `ValueError`; the new contract stores the type first (change #2).

References.

[1] Lecture 3 of this course: `presentation_03_option_basics_and_bsm_model_2026.ipynb` (forwards, calls, puts, parity, Black–Scholes PDE and formula, Greeks).

[2] Lecture 5 of this course: `notebook_building_blocks.ipynb` and the `solution` folder (the library).

[3] Lecture 4 of this course: `Tree.ipynb` (risk-neutral pricing in the tree; literature: Shreve I).

[4] J. C. Hull, *Options, Futures, and Other Derivatives*, Pearson: chapter "Trading Strategies Involving Options" (bull spreads) and chapter "Exotic Options" (binary options: cash-or-nothing call $Q\,e^{-rT}N(d_2)$).

[5] S. E. Shreve, *Stochastic Calculus for Finance II: Continuous-Time Models*, Springer, 2004, Ch. 5 (risk-neutral pricing); and Vol. I, Ch. 1 (no arbitrage), the literature of Lecture 4.

[6] D. T. Breeden, R. H. Litzenberger, Prices of state-contingent claims implicit in option prices, *Journal of Business* 51(4), 1978, 621–651 (the strike derivative of the call price as the price of a digital).